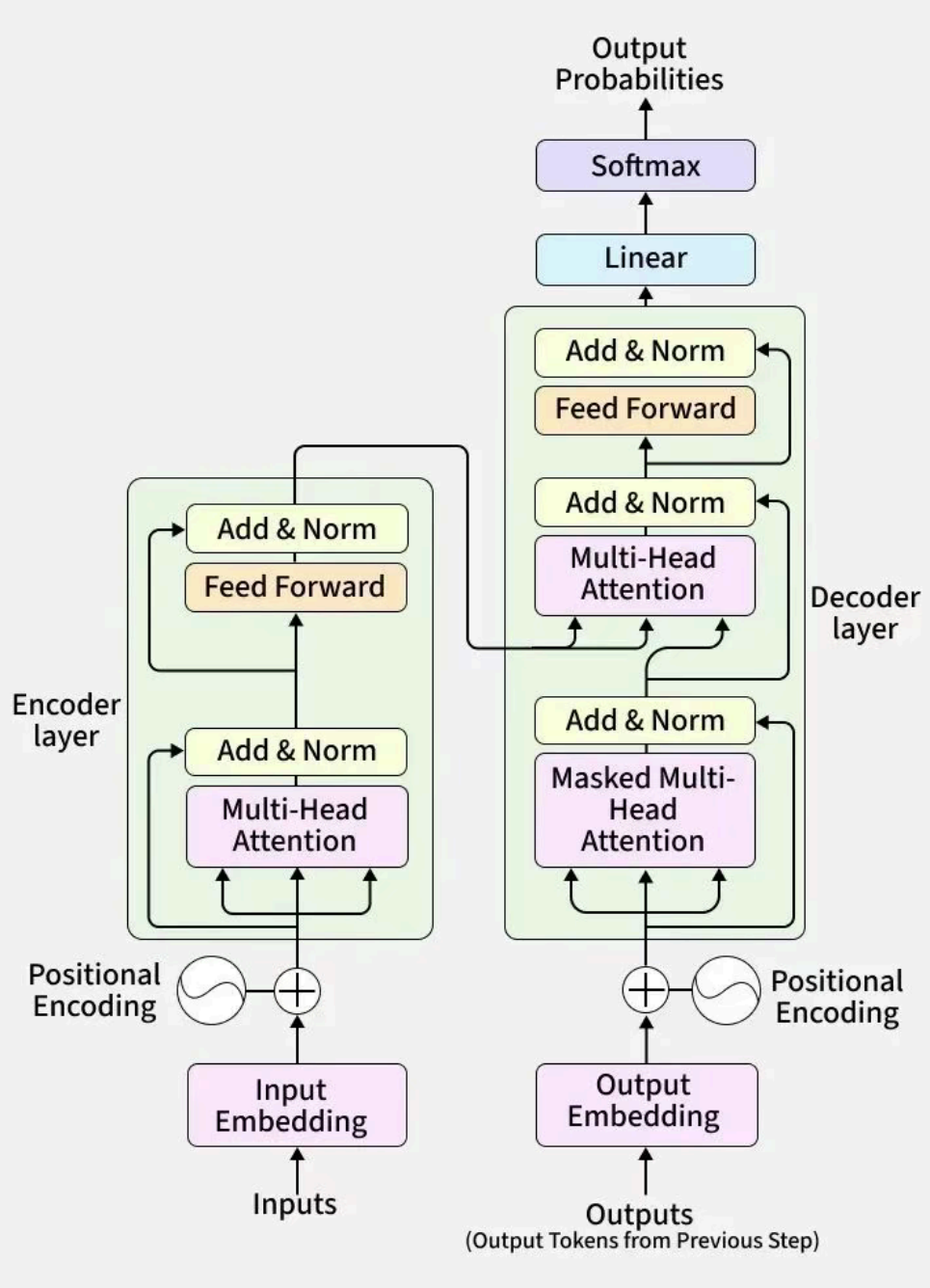
AI Infra入门干货总结：大模型是如何高效推理的

原创 腾讯程序员 腾讯技术工程 2026年5月25日 02:36 广东 626人



作者：binnnliu

> 看了很多的文章和视频，我以为我理解大模型的工作原理了，直到看了vLLM的代码，我发现很多地方理解的太过表面。因此花了大概2个月的业余时间，深入阅读了vLLM的源码，本文算是对于学习代码的一个总结。另外由于当前主流LLM都是 Decoder-Only 架构，本文会聚焦LLM，不会像网络上其他介绍Transformers的文章从原始论文的Encoder-Decoder架构讲起。



在阅读 vLLM 源码的过程中发现，追踪推理过程中每一步的张量维度变化，对于理解大模型工作原理非常有用。因此，本文将以 Llama 3为例，介绍推理过程中的每个计算环节，在每个计算环节都会标注Tensor的维度变化。

我们知道通过批处理可以提高GPU的利用率：本质是提升计算强度，即通过复用权重数据来均摊内存访问开销；而减少Kernel启动开销与通过海量warp充分发挥隐藏延迟仅是随之而来的次要优化。而批处理内的请求要步调一致，同时开始，同时结束。而对于LLM这类生成式任务，其核心矛盾在于每个请求的输出序列长度是不可预测且差异巨大的。那怎么办呢?

有没有可能将调度的从request level下沉到token level呢？ 恭喜你发明了continuous batching。

那每个请求的KV Cache显存申请是不是应该也是token level，不要一次申请所有的显存。搞一个地址数组(block table)来维护每个请求的KV Cache地址就好？ 恭喜你发明了Paged Attention。

没错，以上两个概念是当今大模型得以高性能运行的关键。

连续批处理 (Continuous Batching)

在vLLM调度器的视角中，不存在Prefill 阶段和Decode阶段的区分。 每个请求主要关注：

- num_computed_tokens：已经计算过的 token 数（含 Prefix Cache 命中的部分）
- num_tokens：该请求当前总共拥有的 token 数（prompt + 已生成的 output）

每一步调度的目标就是：让 `num_computed_tokens` **追上** `num_tokens` 。差多少就调度多少——当然要受限于 token 预算。

调度受4个硬性约束限制：

1. 最大并发请求数

   `self.max_num_running_reqs = scheduler_config.max_num_seqs`

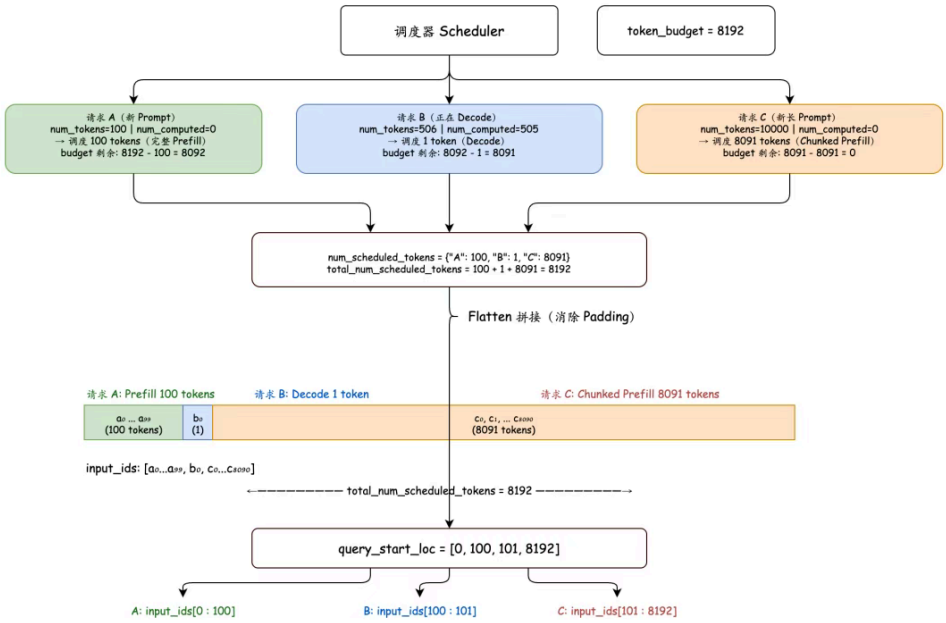
2. token budget　单步最多计算多少 token（所有请求之和），控制 GPU 计算量上限

```
self.max_num_scheduled_tokens = (
    self.scheduler_config.max_num_scheduled_tokens       # 优先用这个
    if self.scheduler_config.max_num_scheduled_tokens
    else self.scheduler_config.max_num_batched_tokens    # fallback
)
```

3. 模型最大序列长度

   `self.max_model_len = model_config.max_model_len`

4. 是否还有空闲的KV Cache blocks

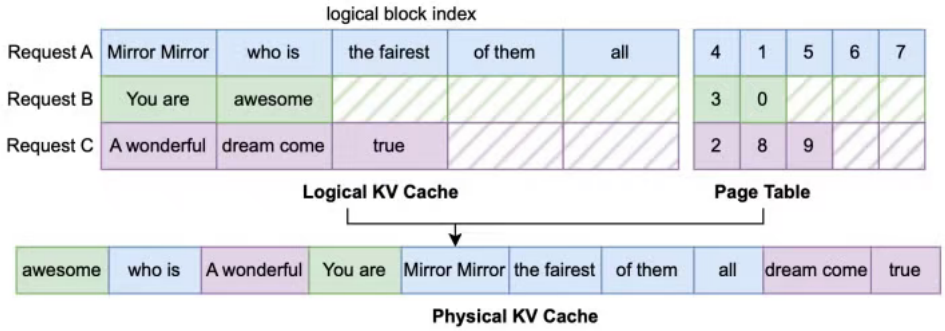


如上图所示，vLLM的每轮推理都会基于token level调度，后续的执行的输入都是类似input_ids这种打平的token数组，即Flattened。(这里的调度逻辑是在Tokenize之后，本文后续的num_sched_tokens都对应上图中的total_num_scheduled_tokens)

Paged Attention

vLLM启动时会申请显存 kv_cache shape为：`[num_layers, 2, num_blocks, block_size, num_kv_heads, head_dim]`，每一层key_cache和value_cache的shape都是`[num_blocks, block_size, num_kv_heads, head_dim]`。

block_table会记录每个请求分配到的物理块ID，shape为`[max_num_reqs, max_num_blocks_per_req]`，请求0分配了块`[5, 8, 12]`，请求1分配了块`[3, 7]`



> 为什么 `slot_mapping` 和 `block_table` 不需要 `num_layers` 维度？如果一个请求的第 25 个 Token 被分配到了物理块 ID 为 `8` 的位置，那么在模型的**第 0 层到第 31 层**，这个 Token 的 KV 值都会被存储在物理块 `8` 的相同位置。

而推理过程中，通过slot_mapping告诉kernel 把新产生的 KV 写到哪个 slot，通过block_table告诉 kernel 去哪些物理 block 读取 KVCache。

```
# slot_idx对应token对应的最终存储位置：  slot_idx = block_idx * block_size + block_offset
const int64_t slot_idx = slot_mapping[token_idx];

const int64_t block_idx = slot_idx / block_size;     // 计算块索引
const int64_t block_offset = slot_idx % block_size;  // 计算块内偏移
```

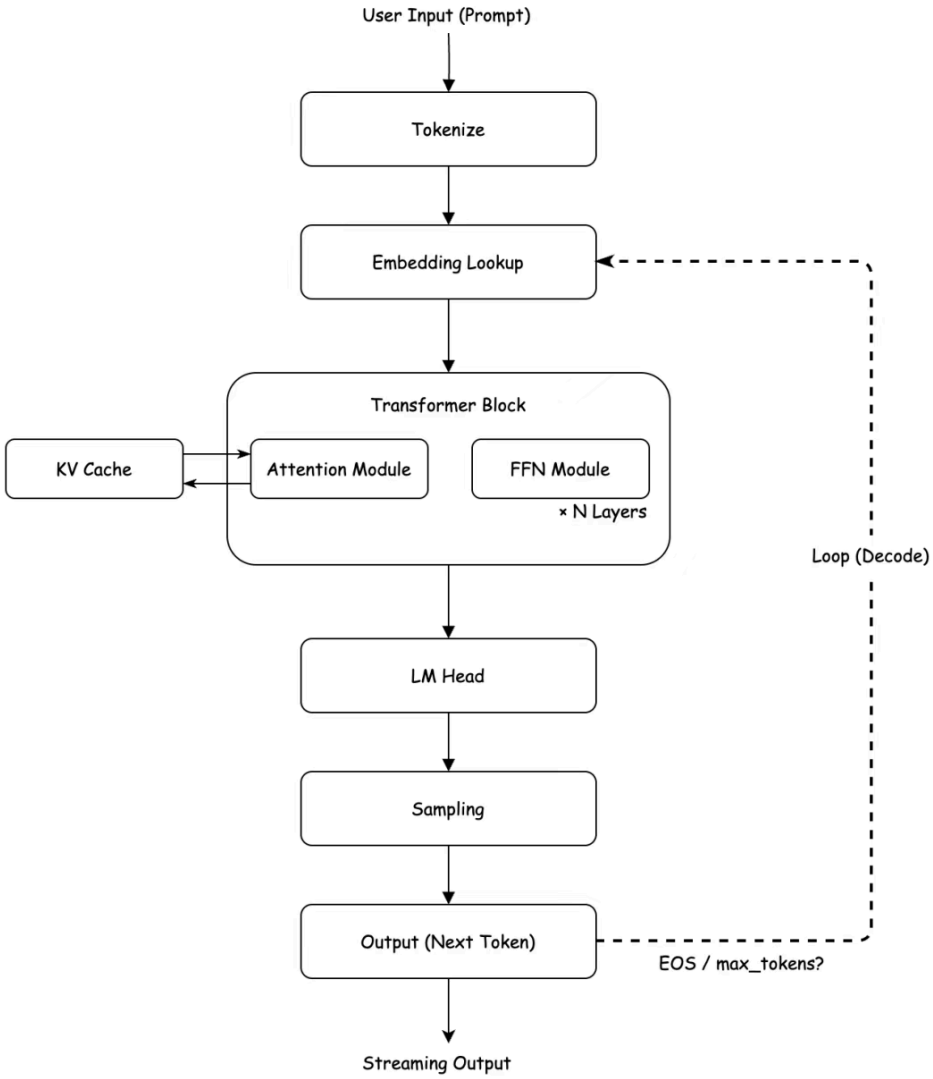
> 比如：`block_size = 16`（每个块存储16个token）,请求0的block_table为：[5, 8, 12]，token位置为25，计算过程：
>
> 1. block_idx = 25 // 16 = 1（属于第1个虚拟块）
> 2. block_idx = block_table[0, 1] = 8（从页表查找物理块ID）
> 3. block_offset = 25 % 16 = 9
> 4. slot_idx = 8 * 16 + 9 = 137

PagedAttention 的虚拟页表机制解决了显存碎片的问题，极大地提升了 GPU 的显存利用率，是支撑 Continuous Batching 高性能推理的基础。

然而PagedAttention 引入的 block_table 间接寻址机制，打破了一个请求在物理显存上的绝对连续性。当 Attention Kernel 跨越 block 读取历史 KVCache 时，会触发离散访存（Uncoalesced Access），这在底层对 Memory Controller 是非常不友好的，同时也会导致 L2 Cache 的命中率下降，带来一定的带宽折损。当然，系统设计从来都是 Trade-off。

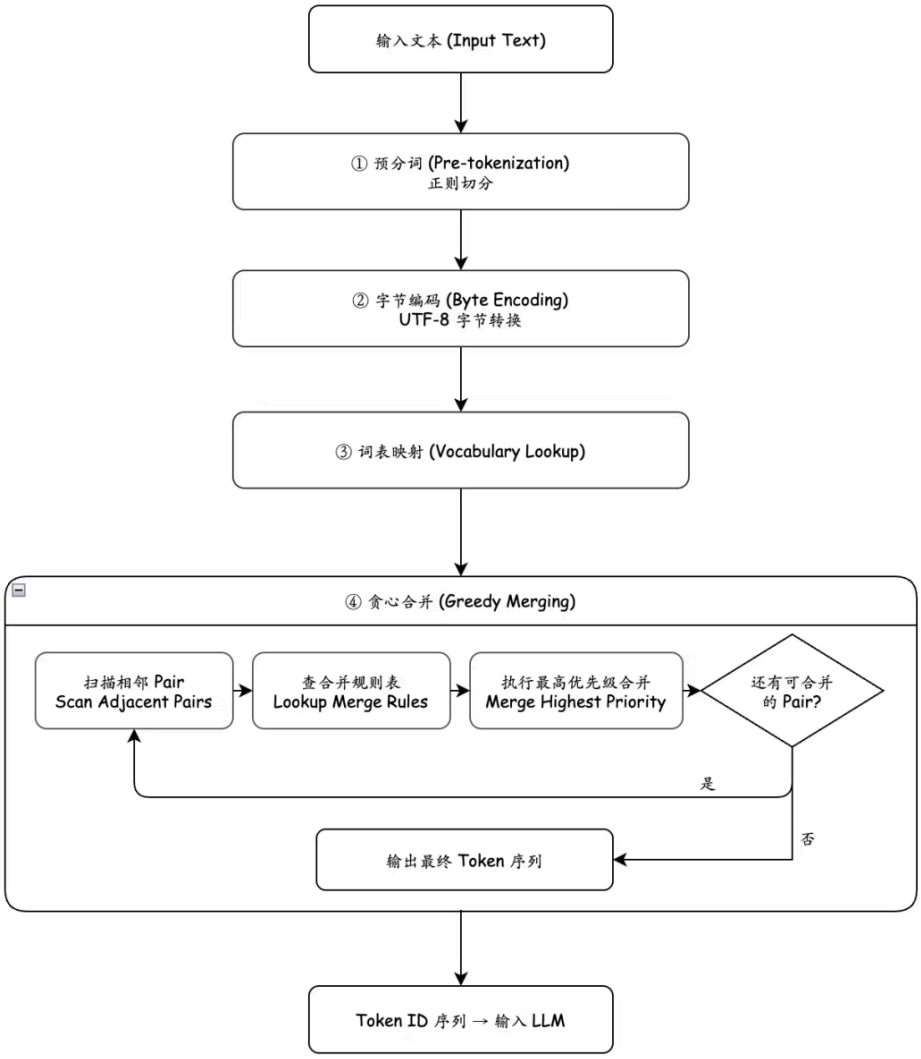
vLLM 通过设置合理的 block_size（默认通常是 16）来缓解这个问题：在一个 block 内部的 16 个 Token 的物理显存依然是连续的，能够保证高效率的合并访存。相比于因显存碎片导致的OOM，牺牲少部分的访存带宽换取整个系统吞吐量（Throughput）的大幅跃升，在 LLM 推理的整体逻辑中是划算的。

LLM推理流程

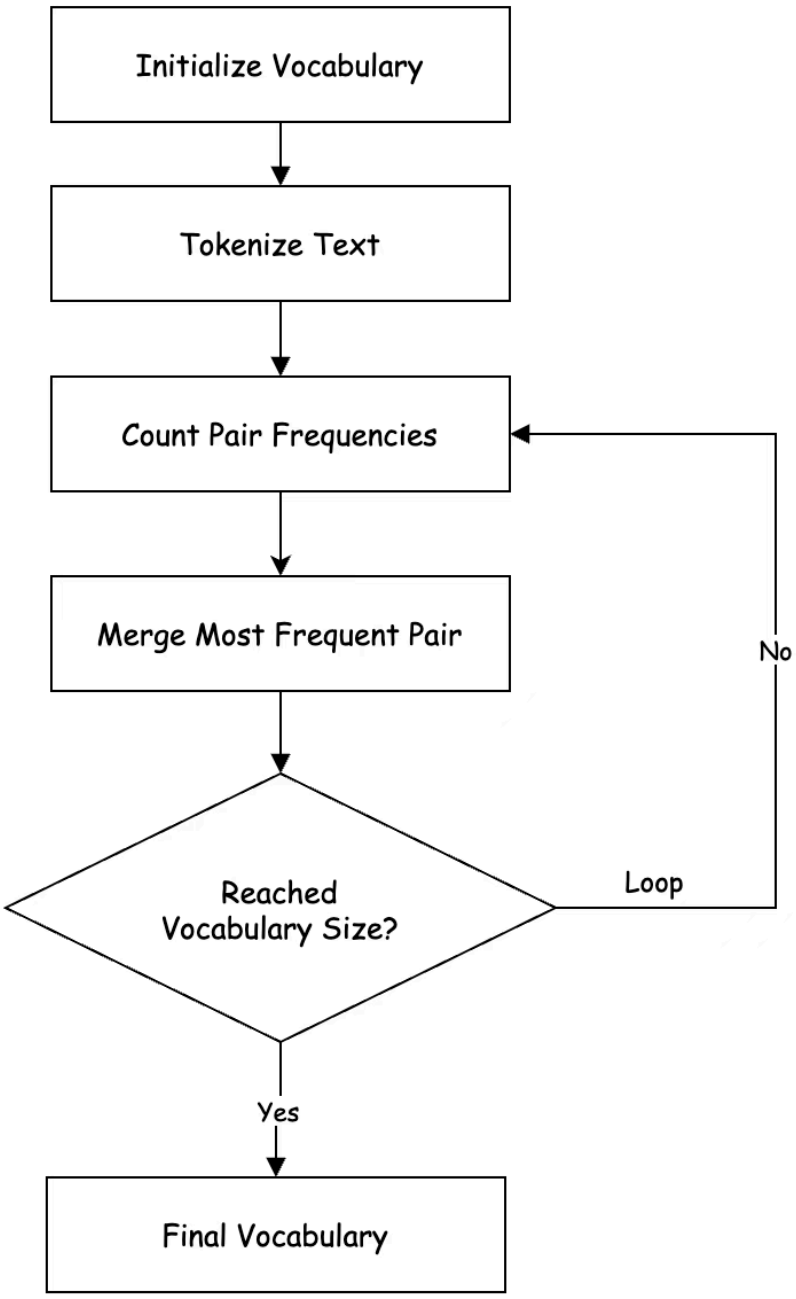


Tokenize

对用户输入提示词进行分词并转换为数字表示。目前主流 LLM 几乎全部使用 BPE（Byte Pair Encoding）。



BPE的词表训练方式如下：



Embedding Lookup

Embedding Lookup 本质上是一个查表操作，它将一维的 Token ID 数组 `[num_sched_tokens]` 直接转化为 `[num_sched_tokens, hidden_size]` 的特征矩阵，作为进入大模型真正意义上的数学输入。

理论实现

目标矩阵	计算公式	输入 X_{ids} 维度	权重矩阵 W_{emb} 维度	输出结果维度	备注
Embedding	$E = W_{emb}[X_{ids}]$	`[batch_size, seq_len]`	`[V, hidden_size]`	`[batch_size, seq_len, hidden_size]`	本质是索引取值，而非矩阵乘法

vLLM中的实现

目标矩阵	计算公式	输入 X_{ids} 维度	权重矩阵 W_{emb} 维度	输出结果维度	备注
Embedding	$E = W_{emb}[X_{ids}]$	[num_sched_tokens]	[V, hidden_size]	[num_sched_tokens, hidden_size]	Flattened 布局

将 Batch 中所有请求的有效 Token 拼接为一维长向量，彻底消除 Padding。

Transformer Block

Attention Module (以标准GQA + RoPE为例)

RMSNorm

为了保证大模型在神经网络中的数值稳定性，必须引入特征归一化（Normalization）机制。与传统 LayerNorm 要求同时缩放与平移不同，为了提升计算效率，现代大模型广泛为采用了 RMSNorm： 砍掉平移操作，仅沿特征维度对数据进行尺度缩放，能够有效防止前向传播中的方差膨胀与硬件数值溢出，从而极大缓解反向传播时的梯度异常（消失或爆炸）问题，为深层网络的稳定训练提供基础保障。

目标矩阵	计算公式	输入 X 维度	权重 W_{norm}维度	输出结果维度	备注
RMSNorm	$y = \frac{x}{\text{RMS}(x)} \odot W_{\text{norm}}$	[num_sched_tokens, hidden_size]	[hidden_size]	[num_sched_tokens, hidden_size]	W_{norm} 是一维向量而非矩阵，做的是逐通道缩放而非 GEMM

Q/K/V Linear Proj - Fused QKV

目标矩阵	计算公式	输入 X 维度	权重矩阵 W_{qkv} 维度	输出结果维度	备注
Fused QKV	$QKV_{out} = X \cdot W_{qkv}$	[num_sched_tokens, hidden_size]	[hidden_size,qkv_proj_size]	[num_sched_tokens, qkv_proj_size]	权重按列拼接，执行单次宽矩阵 GEMM 4096 (Q) + 1024 (K) + 1024 (V) = 6144

Reshape & Cache

目标矩阵	Q, K, V Slice	RoPE(维度不变)	Multi-Head Reshape	备注
Query (Q)	[num_sched_tokens, hidden_size]	$Q_{rope} = R_{pos} \cdot Q$	[num_sched_tokens,num_heads,head_dim]	QKV_{out} 前 4096 列
Key (K)	[num_sched_tokens, kv_channels]	$K_{rope} = R_{pos} \cdot K$	[num_sched_tokens,num_kv_heads,head_dim]	QKV_{out} 中间 1024 列

目标矩阵	Q, K, V Slice	RoPE(维度不变)	Multi-Head Reshape	备注
Value (V)	`[num_sched_tokens, kv_channels]`		`[num_sched_tokens,num_kv_heads,head_dim]`	QKV_{out} 最后 1024 列

注：$Q_{rope} = R_{pos} \cdot Q$是一个块对角旋转矩阵，实际实现中通过元素级 sin/cos 运算避免矩阵乘法。

这个阶段的大概流程： QKV 按列拆分 -> 对Q、K进行RoPE计算 -> Q、K、V Reshape多头视角 -> 然后reshape_and_cache按照slot_mapping Scatter写入Paged KVCache。

RoPE 的核心思想就是：**用旋转的角度来代表 Token 的位置**。 虽然不是矩阵乘法，sin/cos 属于超越函数，在 GPU 上由 SM 内的特殊函数单元（SFU, Special Function Unit）执行。SFU 的吞吐量通常仅为 FP32 ALU 的 1/4。所以VLLM中不在 RoPE kernel实时计算sin/cos ，而是引擎初始化时按模型配置预计算好 cos_sin_cache，shape 为 [max_position_embeddings, rotary_dim]。 RoPE kernel 运行时按 positions 索引出对应的 cos/sin 行，与 Q/K 做逐元素乘加即可。本质是用一份小表（典型大小几 MB 到几十 MB）换掉每次 forward 几百万次的 sin/cos 调用——经典的空间换时间 /访存换算力。

Attention (FlashAttention, Scaled Dot-Product Attention)

$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$ 读取请求对应 KV Cache 进行 FlashAttention 计算。

> Softmax 的核心原理可以用一句话概括：将一组任意的实数（通常称为 Logits），转化为一套总和为 1 的概率分布，同时放大差异。ps: 这里的d_k为head_dim。

感觉Attention的逻辑是整个LLM推理最复杂的地方了：

1. Attention 的数学逻辑并不复杂，但是为了提升计算强度，打破内存墙，业界在这里做了非常多的优化，比如FlashAttention；
2. 其他阶段基本都是Token维度打平的，而Attention计算必须要在请求维度进行运算。

特别注意: 在进入Attention kernel之前，需要从全局Flattened视角 `[num_sched_tokens, ...]` 切换到请求维度视角 `[batch_size, ...]` 。这是因为Attention计算本质上是请求内部的操作，不同请求的KV Cache不能交叉。vLLM/FlashAttention通过 `cu_seqlens` （累积序列长度）数组来实现这种变长序列的请求级隔离，而无需真正Reshape为 `[batch_size, ...]` 的规整张量。 这里的K和V Shape算子逻辑上为 `[batch_size, num_kv_heads, seq_len, head_dim]` ， 而不同请求的seq_len肯定不一致。实际上在cuda算子层已经确保了不同请求不会分配到同一个 thread block，即保证这里的运算是请求维度隔离的。

K的Shape为 `[seq_lens, num_kv_heads, head_dim]`，包含当前请求所有历史的KCache，seq_lens是所有历史KV对应的Token长度，但物理上，由于Paged Attention机制，这些历史KCache并非连续存储，而是离散分布在HBM的物理块中（即底层实际Shape为[num_blocks, block_size, num_kv_heads, head_dim]）。Kernel在执行时，会通过 block_table 动态间接寻址，在片上SRAM中将其拼凑为逻辑上的连续序列参与计算。

V的Shape为 `[seq_lens, num_kv_heads, head_dim]`，包含当前请求所有历史的VCache

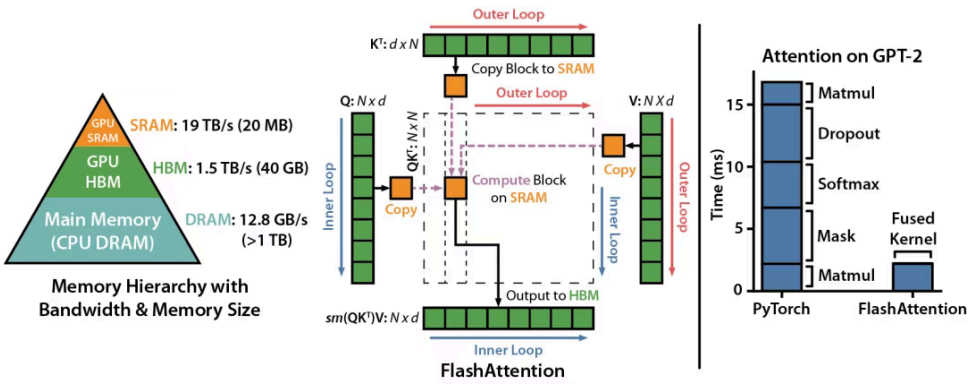
Q的Shape为 `[query_lens, num_heads, head_dim]`，query_lens是本次Q对应的Token长度

操作名称	数学公式	逻辑说明 / 目的
GQA分组共享	1 KV Head 映射给 4 Q Head	把KV的Shape从`[seq_lens, num_kv_heads, head_dim]`变为`[seq_lens, num_heads, head_dim]` 逻辑上等价于将1个KV Head的数据广播给4个Q Head；物理实现中通过索引映射kv_head_idx = q_head_idx // (num_heads // num_kv_heads)避免显存复制。
Q-K Dot Product	$S = Q \times K^T$	查询Query和该请求所有Key的点积，注意力原始分数。->`[query_lens, num_heads, seq_lens]`
Scale	$S = S/\sqrt{d}$	对原始分数进行数值缩放（d 为向量维度），防止 Softmax 函数进入梯度极小的区域。
Mask	$S = S + M$	注入因果逻辑，通过加上掩码矩阵 M（通常将未来位置设为 $-\infty$），阻止模型“看到”未来的信息。
Softmax	$P = \text{softmax}(S)$	将处理后的分数转化为总和为 1 的注意力权重（概率）。->`[query_lens, num_heads, seq_lens]`
MatMul	$O = P \times V$	根据注意力权重 P 对值向量（Value）进行加权求和，得到最终的**输出特征表示**。 V的Shape为`[seq_lens, num_heads, head_dim]` -> `[query_lens, num_heads, head_dim]`

Attention 的数学逻辑非常清晰，但在大模型推理（尤其是长文本 Prefill 阶段）中，如果严格按照数学公式分步执行，会在 GPU 的 HBM（全局显存）中产生庞大的中间矩阵 S（注意力分数）和 P（Softmax概率），其 $\mathcal{O}(N^2)$ 的内存访问量会严重影响性能，即所谓的内存墙问题。

为了打破这个瓶颈，vLLM 默认采用 FlashAttention 及其变体（如 Flash-Decoding）。FlashAttention 的核心思想是 Kernel 融合 (Kernel Fusion) 与 分块计算 (Tiling)。但分块计算面临一个致命的数学障碍：Softmax 操作需要知道全局的最大值和指数和才能计算，这通常要求完整的中间矩阵驻留在显存中。FlashAttention 巧妙地引入了 **Online Softmax** 算法，通过维护局部的当前最大值（Running Max）和当前指数和（Running Sum），并在读入新块时利用动态缩放因子（Rescaling Factor）对旧结果进行修正。

正是凭借 Online Softmax，FlashAttention 成功解除了全局数据依赖，将从 Q-K 点积、Softmax 到乘 V 的整个逻辑步骤，融合成了一个单一的 CUDA Kernel。在物理执行上，它一块一块地在 SRAM 中完成计算，彻底避免了将庞大的中间矩阵 S 和 P 读写 HBM，从而大幅打破了内存墙限制。



Prefill 阶段，seq_lens 和 query_lens 都是num_prompt_tokens。当前 Prompt 计算出的 KV 向量先被 reshape_and_cache_flash 按 slot_mapping 写入 Paged KV Cache，随后 attention kernel 通过block_table 从 cache 中读取 K/V 完成本层 attention 计算；这份写入同时也作为该序列的 KV Cache，供后续 decode 步骤复用。

> vLLM v0中prefill和decode 走两条独立分支：prefill 通过 `attn_metadata.is_prompt == True` 调 `flash_attn_varlen_func`，kernel直接读临时的物理上连续的HBM buffer 里的 K/V，不读 cache；decode 走 `paged_attention_v1/v2`，K/V 全部从 cache 读。v1 把两路统一成一次 `flash_attn_varlen_func` 调用，K/V 全部走 paged cache + block_table，代价是 prefill 多一次"写 cache → 读 cache"的间接寻址，换来对 prefix cache / chunked prefill 的天然支持。只有v0版本 FlashAttention不读KV Cache，因为 v0 没有 prefix cache / chunked prefill。

> V1中即便某些 backend（如 FlashInfer、MLA-sparse）把混合 batch 物理拆分成prefill / decode 两个独立 kernel 调用,prefill 仍然从 paged cache 读 K/V - 因为 v1 需要支持 prefix cache 和 chunked prefill 。

Decode阶段，seq_lens为已计算的历史 token 总数， query_lens为1。

对比Prefill和Decode，可以发现：

- Prefill阶段：QKV_Proj、Attention、O_Proj、MLP 本质上都是稠密矩阵乘法（GEMM），其实就是通过请求内token-level计算复用模型权重，因此算术强度极高，使得 GPU 在 Prefill 阶段主要受限于 Tensor Cores 的物理算力峰值，属于计算密集型（Compute-bound）。
- Decode阶段：由于自回归特性，每次只处理一个 Token（ $\text{query_lens} = 1$ ），若无优化，此时QKV_Proj、Attention、O_Proj、MLP全都退化为矩阵向量乘法（GEMV）。虽然计算量不大，但需要把巨大的 KV Cache和模型权重 从 HBM 搬运到 SRAM，非常吃显存带宽，因此Decode阶段是典型的访存密集型（Memory-bound）。

而 Continuous Batching 的价值在于在Prefill实现请求内复用模型权重的基础上，进一步通过token-level的调度让多个请求复用模型权重，将 QKV_Proj、O_Proj、MLP 重新变回矩阵乘法，极大地摊薄了读取权重的显存开销。

但是Attention则比较特殊，由于每个请求必须读取自己独占的 KV Cache（无法跨请求共享），而且Decode每次只处理一个 Token，Attention 计算始终无法均摊访存开销。此外，由于当前 Query 必须与所有历史 Key/Value 交互，哪怕有KVCache的存在，其计算与访存量也是随着上下文长度 L 呈 $O(L)$ 线性膨胀。这里特别注意其实Prefill阶段每次处理num_prompt_tokens个Token，Attention 计算是可以均摊访存开销的。

ps: 所以其实我们日常所说的Prefill是Compute-bound，Decode是Memory-bound，主要是基于 1. 从 HBM 读取模型权重的访存摊销， 2. 从 HBM 读取模型KV的访存摊销。

Multi-Head Concatenation

操作名称	逻辑说明	vLLM 输入维度	输出结果维度	备注
Concat / Flatten	消除多头维度，恢复隐藏层维度	`[num_sched_tokens, num_heads, head_dim]`	`[num_sched_tokens, hidden_size]`	物理内存通常已经是连续的，这里本质上是一个改变 Tensor View 的操作。 (hidden_size = num_heads * head_dim)

O_Proj - 线性融合 (Output Projection)

目标矩阵	计算公式	输入 X_{concat} 维度	权重矩阵 W_o 维度	输出结果维度	备注
O_Proj	$O_{out} = X_{concat} \cdot W_o$	`[num_sched_tokens, hidden_size]`	`[hidden_size, hidden_size]`	`[num_sched_tokens, hidden_size]`	跨 Head 的信息交互与融合。这也是一次标准的稠密矩阵乘法（GEMM / GEMV）。

Residual Connection

不要推翻重来，而是在原有的基础上学习“修正值”(Delta)。如果没有 ResNet 证明网络可以堆到 100 层以上且不梯度消失，后来的 BERT (24层) 和 GPT (96层+) 根本不敢设计得那么深。**它是“Scaling Law”在架构上的物理基础。**

目标矩阵	计算公式	输入 X_{orig} 维度	输入 O_{out} 维度	输出结果维度	备注
Residual Add	$X = X_{orig} + O_{out}$	`[num_sched_tokens, hidden_size]`	`[num_sched_tokens, hidden_size]`	`[num_sched_tokens, hidden_size]`	X_{orig} 是**进入这一层 RMS Norm 之前**的原始输入。将注意力机制提取的新特征“加回”主干数据流中。

实际vLLM中，Residual Add会和下一步的RMSNorm融合成一个 CUDA kernel。

FFN

RMSNorm

对 x 再进行一次归一化。在 vLLM 中，通常与上一层残差相加（Residual Add）融合，减少 HBM 读写。

Gate_Proj & Up_Proj

通常 intermediate_size 是 hidden_size 的 4 倍左右 (或 8/3 倍)。

目标矩阵	计算公式	输入 X 维度	权重 W 维度	输出维度
Gate Proj	$X_{gate} = X \cdot W_{gate}$	`[num_sched_tokens, hidden_size]`	`[hidden_size, intermediate_size]`	`[num_sched_tokens, intermediate_size]`
Up Proj	$X_{up} = X \cdot W_{up}$	`[num_sched_tokens, hidden_size]`	`[hidden_size, intermediate_size]`	`[num_sched_tokens, intermediate_size]`

vLLM 工程实现（Fused Gate/Up）

为了减少 GPU 核函数启动（Kernel Launch）次数并充分利用内存带宽，vLLM 在加载模型时会将 W_{gate} 和 W_{up} 按列拼接。这样原本两次中等规模的矩阵乘法，就变成了一次宽矩阵 GEMM。

目标矩阵	计算公式	输入 X 维度	权重 Wgate_up 维度	输出维度
Fused Gate/Up	$X_{fused} = X \cdot W_{gate_up}$	[num_sched_tokens, hidden_size]	[hidden_size, 2*intermediate_size]	[num_sched_tokens, 2*intermediate_size]

Activation

SiLU(Gate) * Up 。vLLM 使用 silu_and_mul kernel 在片上（SRAM）直接输出点乘结果，极大节省了显存带宽。

操作名称	数学公式	物理实现说明
Slice & Act	$X_{act} = \text{SiLU}(X_{gate}) \otimes X_{up}$	X_{gate} 为 X_{fused} 前一半，X_{up} 为后一半。

Down_Proj

目标矩阵	计算公式	输入 X_{act} 维度	权重 W_{down} 维度	输出结果维度
Down Proj	$O_{ffn} = X_{act} \cdot W_{down}$	[num_sched_tokens, intermediate_size]	[intermediate_size, hidden_size]	[num_sched_tokens, hidden_size]

Residual Connection

x = x + FFN_Output

```python
class LlamaMLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        # 通常 intermediate_size 是 hidden_size 的 4 倍左右 (或 8/3 倍)
        self.gate_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=False)
        self.up_proj   = nn.Linear(self.hidden_size, self.intermediate_size, bias=False)
        self.down_proj = nn.Linear(self.intermediate_size, self.hidden_size, bias=False)
        self.act_fn = nn.SiLU()

    def forward(self, x):
        # 1. Gate 和 Up 分别进行 Linear 变换
        # 2. Gate 经过 SiLU 激活
        # 3. 两者相乘 (Element-wise multiplication) -> 这就是 GLU (Gated Linear Unit)
        # 4. 最后经过 Down 投影回原维度
        return self.down_proj(self.act_fn(self.gate_proj(x)) * self.up_proj(x))
```

LM Head

将隐藏层特征映射到整个词表空间，输出未归一化的原始得分，即 **Logits**。

在前面的 Transformer Block 中，Prefill 阶段的 `num_sched_tokens` 是所有 Prompt 长度的总和。但实际上vLLM LM Head只需要每个请求序列的最后一个 Token 的特征用来预测下一个词，因此进入 LM Head 前会执行一次 Gather 操作。（Speculative Decoding、prompt_logprobs除外）

> 更近一步，Prefill 阶段 Transformer Block 的最后一层，attention 输出是 [num_sched_tokens, hidden_dim]，但实际上只需要最后一个 token 的 hidden_state 来做 logits 计算（因为之后没有更多的 layer 需要完整的 hidden_states 了）。所以理论上最后一层的MLP（以及 LayerNorm）可以只对需要采样的 token 计算，节省大量计算。如果只需要最后一个 token 的生成结果，那么在最后一层（Last Layer）：
>
> 1. 必须要算的部分：所有 token 的 K 和 V（因为这层的 KV cache 要存下来，供后续 Decode 阶段使用）。
> 2. 理论上可以只算最后一个 token：Q 的投影、Attention 的计算（其实退化成了 Decode 阶段的 Flash-Decoding 计算）、LayerNorm、以及庞大的 MLP。

但是vLLM中最后一层还是老老实实把所有 query_lens 都算了一遍，本质上是工程与物理极限的 Trade-off。

RMSNorm

最后再来一次归一化（防止数值过大）。

Linear (LM Head)

目标矩阵	计算公式	输入 X_{last} 维度	权重矩阵 W_{lm} 维度	输出结果维度	备注
LM Head	$Logits = X_{last} \cdot W_{lm}$	`[num_reqs, hidden_size]`	`[hidden_size, vocab_size]`	`[num_reqs, vocab_size]`	`num_reqs` 即当前 Batch 中的请求数量。以 Llama-3-8B 为例，`vocab_size` 高达 128256，这是一个极庞大的矩阵，vLLM 通常会使用张量并行（TP）按列切分该权重以加速计算。

Sampling

保存原始 Logprobs

如果用户请求了 logprobs（对数概率），Sampler 会在任何处理之前，先对原始 logits 做一次 `log_softmax` 快照保存。这样返回给用户的概率分布反映的是模型的真实输出，不受后续 penalties / temperature 等采样参数的干扰。

```
{
  "token": "Hello",          // 实际被选中的 token
  "logprob": -0.01523,       // 该 token 的对数概率（log_softmax 的结果）
  "bytes": [72, 101, ...],   // token 的 UTF-8 字节表示
  "top_logprobs": [          // 概率最高的前 N 个 token（用户通过 logprobs=N 指定）
    {
      "token": "Hello",
      "logprob": -0.01523    // ln(0.985) ≈ -0.015, 即 98.5% 概率
    },
    {
      "token": "Hi",
      "logprob": -4.32185    // ln(0.013) ≈ -4.32, 即 1.3% 概率
    }
  ]
}
```

Logits Process

在得到原始 Logits 后、进行 Softmax 概率转化前，会经过一系列后处理操作，用于干预最终的生成结果：

- 白名单过滤
  - 结构化输出（JSON schema）：配合 grammar bitmask，每一步动态计算当前合法的 token 集合。
  - 情感分类：只允许输出 `"positive"` / `"negative"` / `"neutral"` 对应的 token。
  - 多选题：只允许输出 `A` `B` `C` `D` 。
- 黑名单过滤
- Non-argmax-invariant Logit Processors
- Repetition Penalty (重复惩罚): 根据历史 Token，对已生成过的词汇扣除分数，防止模型变成“复读机”。

采样

每个请求的采样策略可以是Greedy或Random，由用户设置的 `temperature` 决定

```
@cached_property
def sampling_type(self) -> SamplingType:
    if self.temperature < 1e-5:       # < 0.00001 就是 greedy
        return SamplingType.GREEDY
    return SamplingType.RANDOM
```

1. Greedy 先行：在 temperature 缩放之前，先对当前 logits 执行一次 `argmax` ，保存为 `greedy_sampled` 。

- 如果整个 batch **全是 greedy**（所有请求的 $T \approx 0$），到这里就直接返回，后续步骤全部跳过。
- 这一步必须在 temperature 之前做，否则 greedy 请求的 logits 会被不必要地除以 temperature 导致精度损失。

2. Temperature Scaling：`Logits = Logits / Temperature`。

> $T < 1$: 拉大分数差距。高分越明显，低分被压得更低。模型变得**保守、确定**。
> $T > 1$: 缩小分数差距。分布趋向均匀。模型变得**随机、有创造力**（但也容易胡说八道）。对于 $T < 10^{-5}$的 greedy 请求，此处会用 $T = 1$ 跳过除法（避免除以零），它们的最终结果由 Step 1 的 `greedy_sampled` 决定。

3. Argmax-invariant Processors（如 min_p）：

   min_p是目前唯一的 argmax-invariant 处理器。其规则为：概率不到token最高概率的 min_p倍的 token，全部淘汰。>公式：阈值 = $p_{top} \times min_p$，低于阈值的 token 得分设为 $-\infty$。

> 示例（`min_p = 0.1`，$p_{top} = 0.40$，阈值 = 0.04）：
>
> min_p 的核心价值在于**自适应**：最高概率的 token 自身永远不会被砍（因为 $p_{top} \times min_p < p_{top}$），所以 argmax 结果不变。

   - 模型很确定时（$p_{top} = 0.90$）：阈值 = 0.09，砍得多，几乎只剩最优解。
   - 模型不确定时（$p_{top} = 0.08$）：阈值 = 0.008，砍得少，保留充分的探索空间。

4. Top-K 截断：只保留分数最高的 K 个 Token，其余得分全部设为 $-\infty$。

5. Top-P Masking：按分数从高到低排序并累加 softmax 概率，保留累积概率刚好超过 P 的前若干 Token，尾部 Token 得分设为 $-\infty$。

6. Softmax + Gumbel-Max 采样 确保结果的随机性 公式：$\text{argmax}_i \frac{p_i}{q_i}$，其中 $q_i \sim \text{Exp}(1)$

   Gumbel-Max 用加噪求最大值替代了传统的轮盘赌算法。它将原本需要串行计算前缀和的采样过程，转化为完全独立的 element-wise 并行运算，打破了超大词表下的全局数据依赖。

```
probs = logits.softmax(dim=-1)
q.exponential_()
probs.div_(q).argmax(dim=-1)
```

7. 最终选择：逐请求按 temperature 决定使用哪个结果

```
final = where(temperature < 1e-5,
           greedy_sampled,     # 该请求用贪心结果
           random_sampled)     # 该请求用随机结果
```

output得到 `Next Token ID`，并将其追加到 Input 序列中，进入下一轮循环。

总结

是的，戛然而至。本文主要讲述了经典的大模型推理的全流程，想要真正理解大模型的推理流程理解shape的变化真的很重要。笔者有意克制发散的思维，专注整个推理流程，很多的原理和概念都没有提。后续可以围绕这个基础展开，

比如数学基础：Softmax、RMSNorm原理、RoPE

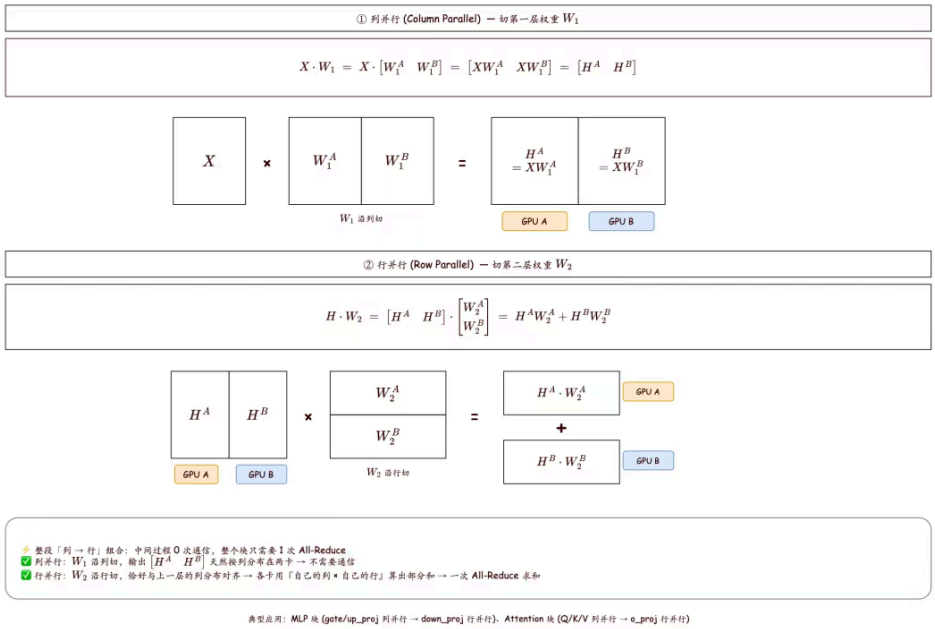
比如Attention计算遇到了内存墙问题

- 导致了 DeepSeek MLA/DSA、线性注意力等的产生，本质是减少KVCache的读取；
- 导致了FlashAttention的产生的，本质上是利用IO Aware、重计算提升计算强度；

比如为什么要MoE scaling law模型不断增大，计算量随参数线性增大，而参数占比的大头又在MLP层，那就把MLP层拆分，美其名曰不同专家，每次执行仅激活部分参数，其本质就是将现有FFN层拆分成多个，然后增加路由，每次可以激活不同专家组。

比如模型太大了，TP、PP、DP、EP、SP、CP，了解了本文的数据流shape变化，TP理解就变的相当容易。

- **Column Parallelism (列切分)：** Fused QKV 和 FFN 的 Fused Gate/Up 都是列切分。例如 QKV 的权重维度在单卡上会变成 `[hidden_size, qkv_proj_size / TP_size]`，输出也相应变成 `[num_sched_tokens, qkv_proj_size / TP_size]`。
- **Row Parallelism (行切分)：** O_Proj 和 FFN 的 Down_Proj 是行切分。权重变为 `[hidden_size / TP_size, hidden_size]`。计算完成后，通过 `AllReduce` 算子来聚合跨卡结果。



比如长上下文时， chunked prefill 到 Flash-Decoding 到 SP 本质是不同维度的prompt的拆分，不过是在解决不同问题，chunked prefill 避免超长 Prompt的新请求影响其他请求，无论是正在处理的 decode 请求还是待处理的其他 prefill 请求， Flash-Decoding 通过 KV 维度的切块（Split-K）提升了 Decode 阶段的 SM（流处理器）利用率，SP (Sequence Parallelism) 则是解决单卡显存塞不下超长上下文的问题。（Flash-Decoding，Sequence Parallelism靠Log-Sum-Exp实现跨 SM或者跨卡的归一化）

再来感叹一句，单从推理来看，大模型的原理其实真没那么高深莫测，最难的数学公式怕是要数RoPE了。但是AI Infra的东西真的多～～ 根本学不完，不像agent方向整天重命名上下文工程🐶 。（提示词工程、function calling、 rag、mcp、memory、agent开发、skills、openclaw、上下文工程、harness工程、hermes）

附1：模型配置参数

标准变量名	数值 (Llama-3-8B)	物理含义说明
hidden_size	4096	隐藏层总维度，也是 Q 向量的总宽度。
num_heads/num_q_heads	32	Query 的头数。决定了注意力机制的并行度。
num_kv_heads	8	Key/Value 的头数。GQA 架构中用于压缩缓存。
head_dim/head_size	128	单个头的维度。公式 $\sqrt{d_k}$ 中的 d_k。
kv_channels/kv_dim	1024	单侧（K 或 V）矩阵的总宽度 (num_kv_heads*head_dim)。
qkv_proj_size	6144	Fused QKV 层的输出总宽度 (hidden_size + kv_channels*2)。
intermediate_size	14336	FFN 内部升维后的维度（通常为 $H \times 8/3$）。
fused_ffn_size/gate_up_proj_size	28672	Fused Gate/Up 层的输出总宽度 (intermediate_size*2)。

标准变量名	数值 (Llama-3-8B)	物理含义说明
vocab_size	128256	词表大小。决定了 LM Head 的输出矩阵尺寸。
num_hidden_layers	32	Transformer Block 的层数。
rope_theta	500000.0	RoPE 旋转位置编码的 base theta 值。

附2：推理运行时变量

标准变量名	数值 (Llama-3-8B)	物理含义说明
num_prompt_tokens	100	prompt 长度
num_tokens	105	prompt + 已生成 output
num_computed_tokens	105	已计算的历史 token 总数
num_sched_tokens	1	本次要计算 1 个新 token（decode），其实就是flatten的query_lens
seq_lens	106	KV Cache 总长度 = 105 + 1
query_lens	1	本次 Q 的长度

附3：推理运行时变量

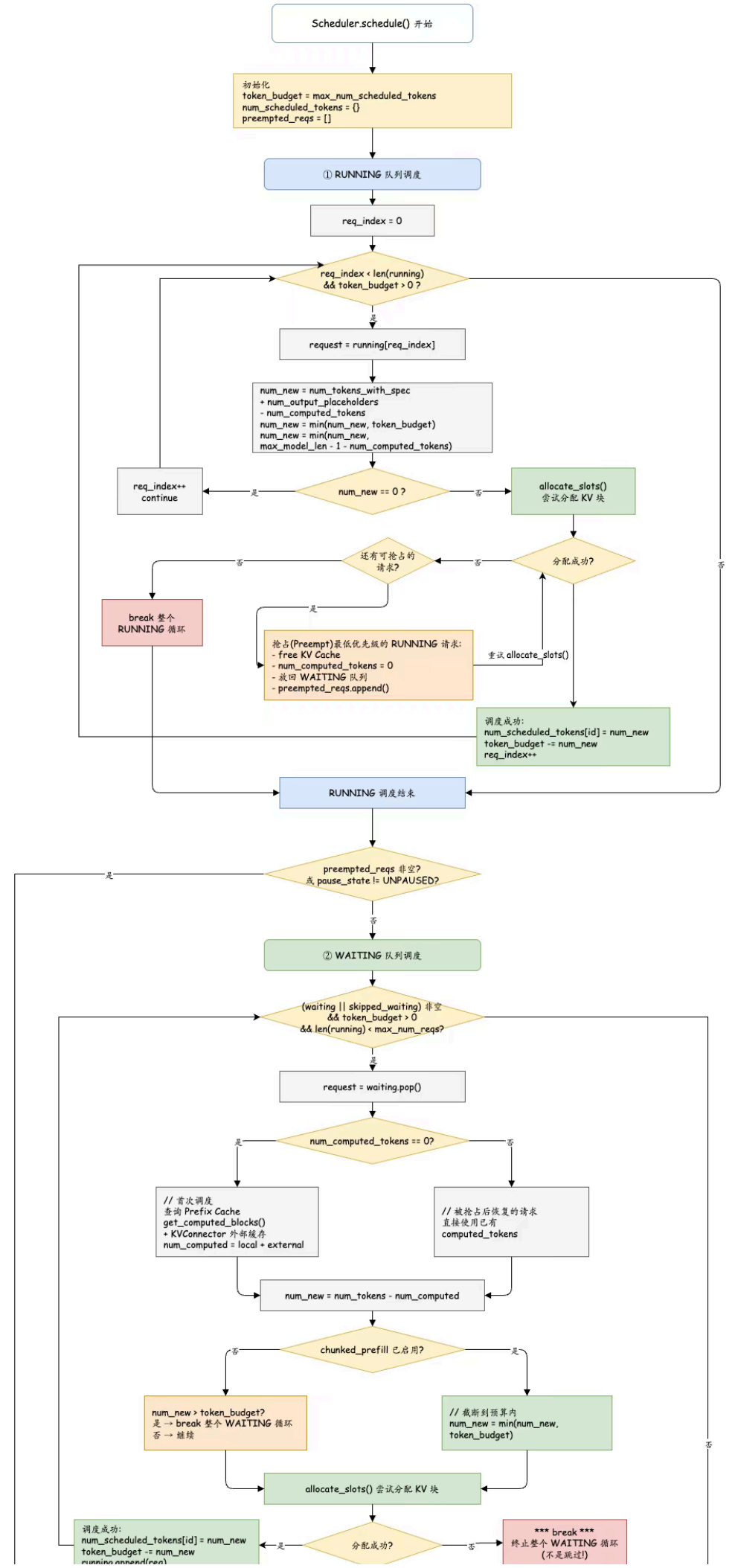
LLM是有多层的，以**Llama3 8B**为例模型的权重组成如下：

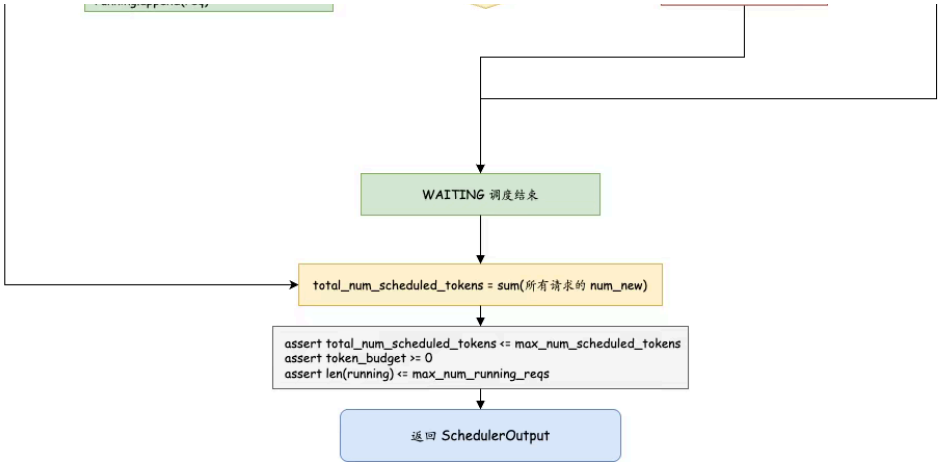
组件	权重名称	参数量维度	占比	本质与说明
Embedding	`embed_tokens.weight`	`[128256, 4096]`	~6.5%	查表矩阵
Attention	`q_proj`	`[4096, 4096]` (×32层)	~15.9%	**4 个 Linear 矩阵，无 bias**。由于 GQA，K/V 矩阵比 Q 小很多。
	`k_proj, v_proj`	`[1024, 4096]` (×32层)		
	`o_proj`	`[4096, 4096]` (×32层)		
MLP	`gate_proj, up_proj`	`[14336, 4096]` (×32层)	~70.1%	**3 个 Linear 矩阵，无 bias**，SwiGLU 激活。**这是模型计算和参数量的绝对大头。**
	`down_proj`	`[4096, 14336]` (×32层)		
Input Norm	`input_layernorm.weight`	`[4096]` (×32层)	< 0.01%	RMSNorm 缩放一维向量。

组件	权重名称	参数量维度	占比	本质与说明
Post-Attn Norm	`post_attention_layernorm.weight`	`[4096]` (×32层)	< 0.01%	RMSNorm 缩放一维向量。
Final Norm	`norm.weight`	`[4096]` (×1)	< 0.01%	最终进入 LM Head 前的 RMSNorm 向量。
LM Head	`lm_head.weight`	`[128256, 4096]`	~6.5%	与 Embedding 维度一致，但**独立存在不共享**。

- **MLP 层**：参数占比 70%，FLOPs 占比也约 70%，是绝对的计算大头。
- **Attention 层**：参数占比仅 16%，但由于需要读取动态生长的 KV Cache，它是**访存带宽**的绝对大头。

附4：vLLM调度详细流程


Scheduler.schedule() 开始
初始化
token_budget = max_num_scheduled_tokens
num_scheduled_tokens = {}
preempted_reqs = []
① RUNNING 队列调度
req_index = 0
req_index < len(running)
&& token_budget > 0 ?
是
否
request = running[req_index]
num_new = num_tokens_with_spec
+ num_output_placeholders
- num_computed_tokens
num_new = min(num_new, token_budget)
num_new = min(num_new,
max_model_len - 1 - num_computed_tokens)
num_new == 0 ?
是
req_index++
continue
否
allocate_slots()
尝试分配 KV 块
分配成功?
否
还有可抢占的
请求?
否
是
break 整个
RUNNING 循环
抢占(Preempt)最低优先级的 RUNNING 请求:
- free KV Cache
- num_computed_tokens = 0
- 放回 WAITING 队列
- preempted_reqs.append()
重试 allocate_slots()
调度成功:
num_scheduled_tokens[id] = num_new
token_budget -= num_new
req_index++
RUNNING 调度结束
preempted_reqs 非空?
或 pause_state != UNPAUSED?
是
否
② WAITING 队列调度
(waiting || skipped_waiting) 非空
&& token_budget > 0
&& len(running) < max_num_reqs?
是
request = waiting.pop()
num_computed_tokens == 0?
是
否
// 首次调度
查询 Prefix Cache
get_computed_blocks()
+ KVConnector 外部缓存
num_computed = local + external
// 被抢占后恢复的请求
直接使用已有
computed_tokens
num_new = num_tokens - num_computed
chunked_prefill 已启用?
否
是
num_new > token_budget?
是 → break 整个 WAITING 循环
否 → 继续
// 截断到预算内
num_new = min(num_new,
token_budget)
allocate_slots() 尝试分配 KV 块
调度成功:
num_scheduled_tokens[id] = num_new
token_budget -= num_new
running.append(req)
是
分配成功?
否
*** break ***
终止整个 WAITING 循环
(不是跳过!)
否



抢占（Preemption)

当 KV Cache blocks 不够分配时，调度器会**抢占最低优先级的 RUNNING 请求**——释放其全部 KV Cache、将 `num_computed_tokens` 重置为 0、放回 WAITING 队列。被抢占的请求下次被调度时需要**重新从头 Prefill**。

> v0 时代，vLLM 有两种 preemption 策略：
>
> - **SWAP**：把被抢占请求的 KV blocks 从 GPU HBM 搬到 host RAM 暂存，资源宽裕时再 swap in 回 GPU。
> - **RECOMPUTE**：直接释放 GPU blocks，重新加入 waiting 队列走prefill 重算一遍。
>
> v1 把 SWAP 路径完全砍掉（`LLM(swap_space=...)` 已废弃并 warn），`Scheduler._preempt_request` 只保留 RECOMPUTE 一条路。原因：
>
> - PCIe 双向搬运 + host RAM 占用的开销，在 FA + 大 batch 下常常比重算一次 prefill 还贵
> - chunked prefill 让重算可以分多步"追上来"，摊平开销
> - 架构上少一条路径，prefix caching / chunked prefill /continuous batching 的交互更干净

```
def _preempt_request(self, request, ...):
    self.kv_cache_manager.free(request)       # 释放 KV Cache
    request.status = RequestStatus.PREEMPTED
    request.num_computed_tokens = 0           # 重置，下次需要重新计算
    self.waiting.prepend_request(request)     # 放回等待队列
```

为什么 RUNNING 阶段分配失败会抢占，而 WAITING 阶段直接终止？

这体现了调度器的核心原则：**已在运行的请求优先级高于等待中的请求。**

- **RUNNING 请求**：已经占用了 KV Cache、消耗了计算资源。如果因为内存不足无法继续，调度器会牺牲优先级最低的 RUNNING 请求（释放其 KV Cache）来保证其他请求能继续推进。这是值得的，因为放弃一个已运行的请求代价很高。

- **WAITING 请求**：还没开始计算，没有沉没成本。如果内存不足，直接停止调度即可，下一个 step 再尝试。

为什么抢占后跳过 WAITING 调度？

发生抢占意味着 KV Cache 已经非常紧张。此时不应该再引入新的 WAITING 请求，因为那会进一步加剧内存压力，可能触发更多抢占，形成恶性循环。

Reference

Attention Is All You Need

Efficient Memory Management for Large Language Model Serving with PagedAttention

Orca: A Distributed Serving System for Transformer-Based Generative Models

Flash-Decoding for long-context inference

SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning

FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision

FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling

AI and Memory Wall

How GPU Computing Works | GTC 2021

Roofline: an insightful visual performance model for multicore architectures

KV Caching in LLMs, explained visually

彻底搞懂KV缓存

KV Caching in LLMs, Clearly Explained

Neural Machine Translation of Rare Words with Subword Units



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号



阅读 2.4万

留言

写留言

还没有留言，来写首评